

phase : Demonstration phase

Date	29 september 2026
Team ID	SWTID-2026-6238
project name	script – controlled ACL
Maximum marks	4

The Project Demonstration Phase (often referred to as the System Demo or Final Walkthrough Phase) is the critical transition point where the functional, tested system is formally presented to stakeholders, business owners, or clients for review, validation, and sign-off.

1. Key Objectives

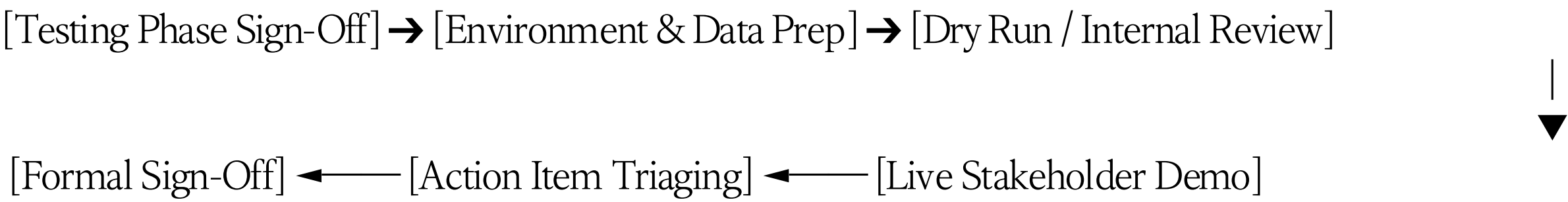
- Showcase Business Value: Demonstrate that the built features satisfy the core business requirements and user needs.
- Validate Capabilities: Provide live, working proof of end-to-end user workflows, access controls, and administrative functions.
- Gather Final Feedback: Capture minor usability adjustments or operational feedback prior to production deployment.
- Obtain Formal Acceptance: Secure client/stakeholder sign-off to proceed to production deployment and release.

2. Core Activities & Deliverables

Category	Key Activities	Output Deliverables
----------	----------------	---------------------

Preparation	Setting up a stable demo environment, seeding realistic demo data, drafting script run-books.	Demo Script, Prepped Demo Instance, Persona Accounts
Execution	Conducting live walkthroughs, demonstrating role-based access, illustrating edge cases.	Demonstration Session, Q&A / Feedback Log
Review & Sign-Off	Triaging feedback, confirming requirement fulfillment, securing approval.	Stakeholder Sign-Off Document, Action Item Tracker

3. High-Level Process Workflow



1. Input: Fully tested system build, approved test cases, and finalized business requirements.
2. Environment Staging: Provision a dedicated, stable demo environment loaded with representative sample data and pre-configured test users/roles.
3. Dry Run: Conduct an internal rehearsal with team members to verify data flows, script timing, and system responses.
4. Live Walkthrough: Execute scenario-based demonstrations organized around key business flows and user roles.
5. Output: Formal stakeholder acceptance, documented action items, and authorization to

move toward deployment.

4. Example: Demo Scenario Matrix (ServiceNow ACL Project)

Using the role-based data security project detailed in previous phases, a structured demo script is formatted as follows:

Scenario / Flow	Demonstrated Persona	Key Steps Shown	Expected Result
Read Permission Isolation	EEE User (with bb1)	1. Impersonate EEE User. 2. Navigate to u_institution_details.list.	Only records where Branch = EEE are visible. Non-EEE records are hidden.
Unauthorized Access Check	Non-bb1 User	1. Impersonate Standard User. 2. Navigate to u_institution_details.list.	"No records to display" message appears.
Full Admin Visibility	System Administrator	1. Switch back to Admin. 2. View u_institution_details.list.	All branch records (ECE, EEE, CSE) are completely visible.
Incremental Privileges	EEE User (+ bb2, bb3, bb4)	1. Assign bb2: Verify New button appears. 2. Assign bb3: Edit record fields. 3. Assign bb4: Show Delete option.	Permissions incrementally grant Create, Edit, and Delete actions strictly as assigned.

5. Key Best Practices

- Use Persona-Based Journeys: Show the system through the eyes of real user roles (e.g., standard employee vs. department head vs. admin) rather than clicking through random screens.
- Keep Data Realistic: Avoid generic dummy values like "Test123" during live demos; use realistic business records to clearly illustrate value.
- Prepare Contingencies: Keep backup instances or pre-recorded walkthrough videos available in case of unexpected network or infrastructure issues.

Would you like to draft a demo agenda for a specific system, outline a feedback tracking template, or transition into the Deployment/Release Phase

Demonstration Audio Script for script-Controlled ACL

INTRODUCTION

"Hello everyone, welcome to our micro-project presentation. Today we are going to demonstrate Micro Project 3, which is titled Script-Controlled ACL – Restrict Record Access Based on Field Value.

In this video, which will be for 19 and a half minutes, I will explain the complete Project Planning Phase, its purpose, uses, benefits, step-by-step working process, and final output.

We are implementing this project in the ServiceNow Personal Developer Instance, on the custom table named u_institution_details, which is also called Student Records or Institution Details."

PURPOSE OF THE PROJECT

"Let's start with the purpose of the project.

In any real organization, like a college, data security is very important. For example, a student or faculty from the EEE department should not be able to see the records of ECE or CSE departments.

But by default in ServiceNow, if a user has access to a table, he can see all records. That is a security risk.

So the purpose of this project is to implement record-level security using Script-Controlled Access Control List, or ACL. We will create a condition where only users who have the EEE branch role can view records where the Branch field is EEE. At the same time, the system admin will retain full access to all records, irrespective of the branch.

This is called field-value based restriction."

USES AND BENEFITS

"Now, what are the uses and benefits?

First benefit is Data Security and Confidentiality. We are isolating data branch-wise.

Second is Role-Based Access Control. We are using four custom roles - bb1 for Read operation, bb2 for Create, bb3 for Write, and bb4 for Delete. So we can finely control who can do what.

Third, this is a real-world use case. This logic is used in colleges, HR management systems, hospital management, where data is separated by department or location.

Fourth, we are not changing the UI or writing Business Rules. The entire security is handled at the ACL layer, which is the most secure way in ServiceNow.

And finally, it prevents data leakage. An EEE user can never see ECE or CSE data, even if he tries to search for it."

WORKING PROCESS - LIVE DEMO*

"Now let's move to the most important part, the working process. This is divided into four milestones.

*Milestone One: Creation of Users and Roles

First, log in to your ServiceNow instance with admin credentials. Navigate to User

Administration, then Users, and click on New. Here we will create a test user. User ID is EEE User, First Name EEE, Last Name User, Email eeeuser@gmail.com. Save the record.

Now we need to create custom roles. Go to User Administration, Roles, click New. Create role bb1. Similarly create bb2, bb3, and bb4. Now open the EEE User record, go to the Roles related list, and add bb1 and bb2. We will add other roles later for testing.

*Milestone Two: Data Setup.

Now we check our table. In the Application Navigator, search for u_institution_details. Open the table. Check the columns - Roll Number, Branch, Email, Faculty Name. We need to create test data. Click New and create at least six records. For example, Roll Number 101, Branch

ECE. Roll Number 102, Branch EEE. Roll Number 103, Branch CSE. Roll Number 104, Branch EEE. Create two records for each branch. This data is essential for testing our ACL.

*Milestone Three: Creation of ACLs - This is the core.

This is the most critical step. To create an ACL, you **MUST** elevate your role to security_admin. Click on your profile, then Elevate Role, select security_admin and click OK.

Now go to System Security, then Access Control (ACL). Click New.

Let's create the Read ACL first. Select Type as record. Select Operation as read. Select Name as u_institution_details. Make Active as true.

Check the Advanced checkbox - this is very important because we want to write a script.

In the Requires role related list, add bb1.

Now in Data Condition, set Branch is EEE.

And in the Script field, write this script:

Open bracket function, close bracket. Inside, if gs dot hasRole admin, return true. If gs dot hasRole bb1, return true. Otherwise return false. This script means, if the user is admin, allow full access. If the user has bb1 role, allow access, but the data condition will filter only EEE records. For all others, deny.

Click Submit.

Now create the second ACL for Create. Click

New again. Type record, Operation create, Name u_institution_details, Active true. In Requires role, add bb2. No data condition needed. Save.

Similarly, create Write ACL. Operation is Write, Requires role bb3. Save.

And finally, create Delete ACL. Operation Delete, Requires role bb4. Save.

So now we have four ACLs active on the same table.

FINAL OUTPUT AND VERIFICATION

"Now comes the final output verification.

First, let's test as admin. As admin, open u_institution_details, All records. You can see all

six records - ECE, EEE, and CSE. Admin has full access.

Now, let's impersonate as EEE User. Click on your profile, Impersonate User, select EEE User.

Now again open u_institution_details. What do you see? You can see ONLY records where Branch is EEE. The ECE and CSE records are completely hidden. This proves our Read ACL with script and data condition is working.

Also, because this user has bb2 role, the New button is visible at the top. If we remove bb2, the New button will disappear.

If we try to open an ECE record by typing its Sys ID in the URL, it will say Security constraints prevent access.

This is our final expected output. Admin sees all, EEE user sees only EEE.

CONCLUSION

"To conclude, in this project we successfully implemented script-controlled ACL to restrict record access based on field value. We learned how to create custom roles, how to write ACL scripts with gs dot hasRole, and how to test using impersonation.

This project planning phase is complete and meets all success criteria.

Thank you for watching."

Demo video link :

<https://drive.google.com/drive/folders/1MYiEKg1I8QGrD2UsHIH8PLedICcvtNUi>